

8

查找

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 静态查找
 - 顺序查找
 - 折半查找
 - 分块查找
- 动态查找
 - 平衡二叉树
 - B树和B⁺树
- 哈希表

目录

1	基本概念和分类
2	静态查找
3	动态查找
4	哈希表

基本概念和分类

- 查找：在数据表中查找满足给定值的第一个记录或全部记录。
- 查找成功：在数据表中查找到满足条件的记录——返回查找结果所在记录的指针。
- 查找不成功：在数据表中没有查找到满足条件的记录——返回空指针。
- 静态查找：没有改变数据表的查找。
- 动态查找：可能需要在数据表中进行插入操作或删除操作的查找。

基本概念和分类

- 关键字：记录中某个成员(数据项)的值，用以识别一个记录。
 - 若关键字可以唯一识别一个记录，则称：主关键字；
 - 若关键字可能识别若干个记录，则称：次关键字。
- 平均查找长度(Average Search Length)

$$ASL = \sum_{i=1}^n P_i C_i$$

- 其中， n 为数据表长度， P_i 为查找数据表中第 i 个记录的概率， $\sum_{i=1}^n P_i = 1$ ， C_i 为找到第 i 个记录时已经进行比较的次数。

目录

1	基本概念和分类
2	静态查找
3	动态查找
4	哈希表

顺序查找

- 顺序查找法：依据顺序查找的方法实现对数据表的查找算法。
- 顺序查找过程：从数据表中的最后一个记录(或第一个记录)开始，逐个比较关键字和给定值，并判断查找是否成功。
- 例：在数据表 $L[1..n]$ 中，顺序查找关键字=key的记录。若找到，返回该记录在数据表中的位置，否则返回0。

顺序查找

- 普通算法

- 遍历整个数组
- 如果找到则返下标
- 找不到则告错

```
for (int i = 0; i < n; i++)  
  
    if (arr[i] == key) {  
        return i;  
    }  
  
return -1;
```

- 哨兵算法

- 设置哨兵
- 遍历整个数组
- 如果找到则返下标
 - 返回越界则为告错

```
arr[n] = key;  
  
int i = 0;  
while (arr[i] != key)  
    i++;  
  
return i;
```



```

int sequentialSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;    // 找到了，立即返回位置
        }
    }
    return -1;          // 遍历完都没找到
}

```

```

int sequentialSearchWithSentinel(int arr[], int n, int key) {
    arr[n] = key;        // 把目标值放在最后一个位置（哨兵）
    int i = 0;
    while (arr[i] != key) { // 少了 i < n 的判断，循环更快
        i++;
    }
    return i;            // 如果 i == n，说明原数组里根本没有
}

```

图的定义和术语

- 顺序查找的时间复杂度 $O(n)$
- 对数据表而言， $C_i = i$ ，即

$$ASL = P_n + 2P_{n-1} + \dots + (n-1)P_2 + nP_1$$

- 在等概率查找的情况下， $P_i = \frac{1}{n}$
- 即顺序表查找的平均查找长度为：

$$ASL = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$$

折半查找算法

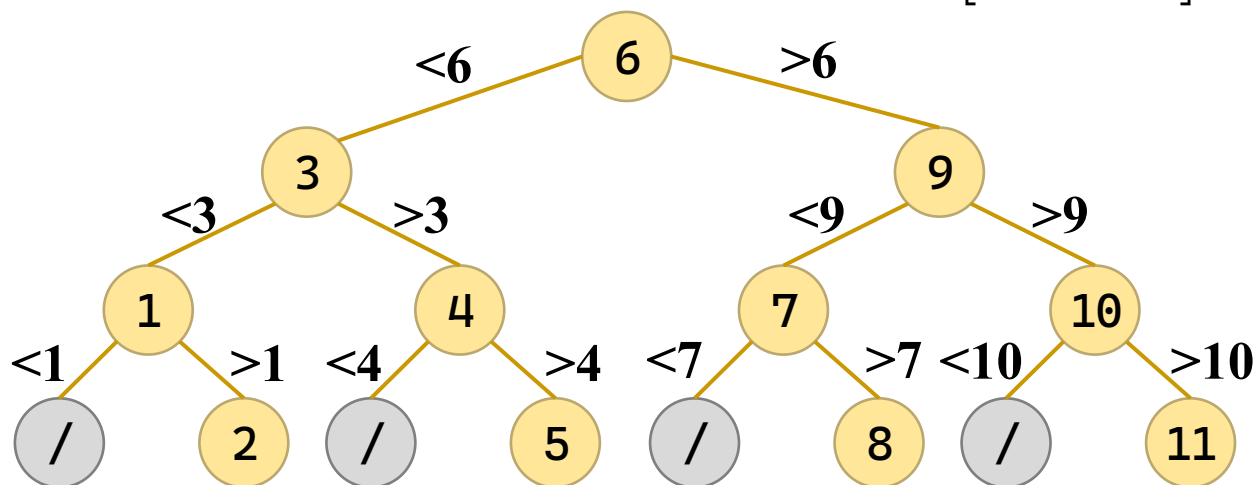
- 有序表：在数据表中，记录按关键字值从小到大递增（或从大到小递减）有序。
- 折半查找法：用折半查找的方法实现对有序表的查找算法，也称为二分查找法。
- 时间复杂度为 $O(\log n)$ ，最多查找次数： $\lfloor \log_2 n \rfloor + 1$

查找长度：1

查找长度：2

查找长度：3

查找长度：4



折半查找算法

• 递归算法

- 如果区间空，退出
- 如果中数找到，
则返回
- 如果中数太小，
找左半边
- 如果中数太大，
找右半边

```
if (low > high)  
    return -1;
```

```
int mid = low + (high - low) / 2;  
if (arr[mid] == key) {  
    return mid;
```

```
} else if (arr[mid] < key) {  
    return binSearch(arr, mid+1, high, key);
```

```
} else {  
    return binSearch(arr, low, mid-1, key);  
}
```

折半查找算法

- 迭代算法

- 如果区间空，退出

```
while (low <= high) {
```

- 如果中数找到，
则返回

```
int mid = low + (high - low) / 2;  
if (arr[mid] == key) {  
    return mid;
```

- 如果中数太小，
找左半边

```
} else if (arr[mid] < key) {  
    low = mid + 1;
```

- 如果中数太大，
找右半边

```
} else {  
    high = mid - 1;  
}
```

```

int binarySearchRecHelper(int arr[], int low, int high, int key) {
    if (low > high) {
        return -1; // 递归出口: 区间无效, 未找到
    }
    int mid = low + (high - low) / 2;

    if (arr[mid] == key) {
        return mid;
    } else if (arr[mid] < key) {
        return binarySearchRecHelper(arr, mid+1, high, key); // 查右半
    } else {
        return binarySearchRecHelper(arr, low, mid-1, key); // 查左半
    }
}

int binarySearchRecursive(int arr[], int n, int key) {
    return binarySearchRecursiveHelper(arr, 0, n - 1, key);
}

```

```

int binarySearchIterative(int arr[], int n, int key) {
    int low = 0;
    int high = n - 1;
    while (low <= high) {
        // 防止 (low + high) 过大导致整数溢出, 用减法替代加法
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) {
            return mid;                // 找到了
        } else if (arr[mid] < key) {
            low = mid + 1;              // 目标在右半区
        } else {
            high = mid - 1;             // 目标在左半区
        }
    }
    return -1;                        // 区间无效, 未找到
}

```

分块查找

- 将数据按照一定原则存入数据表，使数据表划分为若干个逻辑子表。逻辑子表由索引表划分。
 - 索引表：由每个逻辑子表的最大的关键字项构成的有序表。
 - 分块查找：先用顺序查找法在索引表中查找索引记录，再依据该记录值在数据表中的相应逻辑子表查找所需的记录。
- 例如，数据表 $L=\{22, 12, 8, 18, 33, 42, 23, 35, 48, 55, 60, 86, 77\}$ 包含三个逻辑子表。
 - 索引表 $LI=\{22, 1, 4; 42, 5, 4; 86, 9, 5\};$

分块查找

- 索引分块的存储

```
typedef struct {  
    int max;           // 该块内的最大值  
    int start;         // 该块在数组中的起始下标  
    int length;        // 该块的长度（元素个数）  
} IndexBlock;
```

分块查找

- 建立索引表 时间复杂度： $O(n)$

- 遍历块

- 设置起始
 - 设置长度

```
for (i = 0; i < *numBlocks; i++) {
```

```
    index[i].start = i * blockSize;
```

```
    if (i == *numBlocks - 1) // 最后一块可能不满
```

```
        index[i].length = n - index[i].start;
```

```
    else
```

```
        index[i].length = blockSize;
```

- 遍历块内元素

```
    int maxVal = arr[index[i].start];
```

```
    for (j=index[i].start+1;
```

```
        j<index[i].start+index[i].length; j++)
```

- 找出最大值

```
        if (arr[j] > maxVal)
```

```
            maxVal = arr[j];
```

```
    index[i].max = maxVal;
```

```
}
```

分块查找

• 分块查找 时间复杂度： $O(\log b + s)$

- 在索引表中折半查找，定位到块
- 整理块号

```
int low = 0, high = numBlocks - 1, targetBlock = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (index[mid].max == key) {
        targetBlock = mid; break;
    } else if (index[mid].max < key)
        low = mid + 1; // key 比当前块最大值还大，往后找
    else
        high = mid - 1; // key 比当前块最大值小，往前找
}
```

- 遍历块内元素
- 找出值

```
if (targetBlock == -1)
    if (low < numBlocks)
        targetBlock = low;
    else
        return -1; // key比所有块的最大值都大，不存在
```

```
int start = index[targetBlock].start;
int end = start + index[targetBlock].length;
for (int i = start; i < end; i++)
    if (arr[i] == key)
        return i;
```

```

typedef struct {
    int max;           // 该块内的最大值
    int start;         // 该块在数组中的起始下标
    int length;        // 该块的长度 (元素个数)
} IndexBlock;
void buildIndex(int arr[], int n, IndexBlock index[], int*
numBlocks, int blockSize) {
    *numBlocks = (n + blockSize - 1) / blockSize; // 总块数上取整
    for (int i = 0; i < *numBlocks; i++) {
        index[i].start = i * blockSize;
        if (i == *numBlocks - 1) // 最后一块可能不满, 单独处理长度
            index[i].length = n - index[i].start;
        else
            index[i].length = blockSize;
        // 遍历块内元素, 找出最大值 (因为块内完全无序, 必须遍历找最大)
        int maxVal = arr[index[i].start];
        for (int j = index[i].start + 1;
            j < index[i].start + index[i].length; j++)
            if (arr[j] > maxVal)
                maxVal = arr[j];
        index[i].max = maxVal;
    }
}

```

```

int blockSearch(int arr[], IndexBlock index[], int numBlocks,
               int key) {
    // 第一步：在索引表中折半查找，定位到具体块
    int low = 0, high = numBlocks - 1;
    int targetBlock = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (index[mid].max == key) {
            targetBlock = mid;
            break;
        } else if (index[mid].max < key)
            low = mid + 1;           // key 比当前块最大值还大，往后找
        else
            high = mid - 1;         // key 比当前块最大值小，往前找
    }
    // 如果没精确命中，low 指向第一个 max > key 的块，key 就在那一块里
    if (targetBlock == -1)
        if (low < numBlocks)
            targetBlock = low;
        else
            return -1; // key 比所有块的最大值都大，肯定不存在
}

```

// 第二步：在确定的块内进行顺序查找

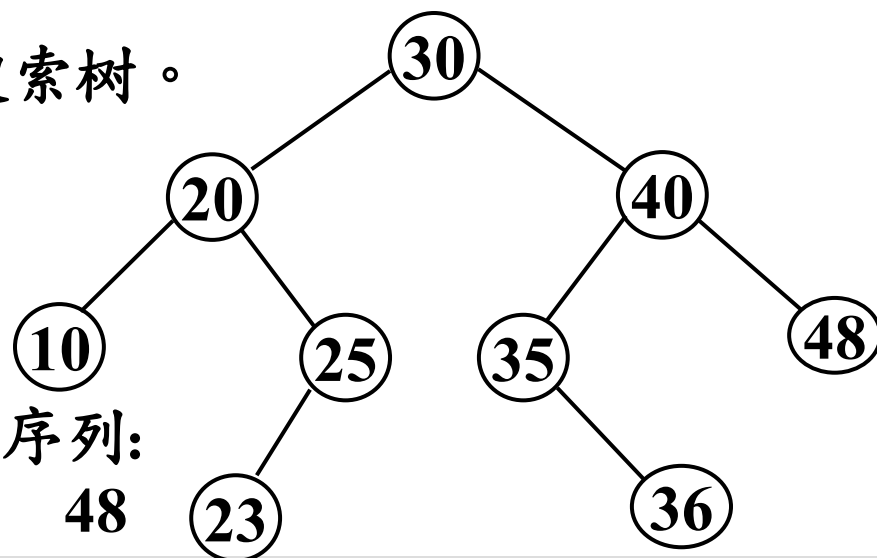
```
int start = index[targetBlock].start;
int end = start + index[targetBlock].length;
for (int i = start; i < end; i++) {
    if (arr[i] == key) {
        return i;
    }
}
return -1; // 块内没找到
}
```

目录

1	基本概念和分类
2	静态查找
3	动态查找
4	哈希表

二叉搜索树

- 定义：或是一棵空树，或是具有如下特性的二叉树：
 - (1) 若左子树不空，则左子树上所有结点的值均小于根结点的值；
 - (2) 若右子树不空，则右子树上所有结点的值均大于(或等于)根结点的值；
 - (3) 左、右子树分别是二叉搜索树。



中序遍历二叉搜索树可以得到排序序列:

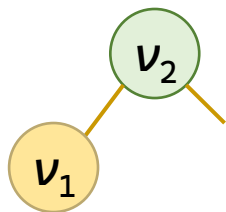
10 20 23 25 30 35 36 40 48

二叉搜索树：插入

• 算法

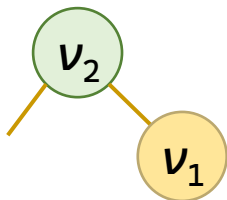
- 如果空树则为根 v_1
- 如果当前值比树根大，

```
if (root == NULL) {  
    return createNode(key);  
}
```



```
if (key < root->data) {  
    root->lchild = insertBST(root->lchild, key);  
}
```

- 如果当前值比树根小，



```
else if (key > root->data) {  
    root->rchild = insertBST(root->rchild, key);  
}
```

```

BSTNode* createNode(int key) {
    BSTNode* node = (BSTNode*)malloc(sizeof(BSTNode));
    node->data = key; node->lchild = NULL; node->rchild = NULL;
    return node;
}

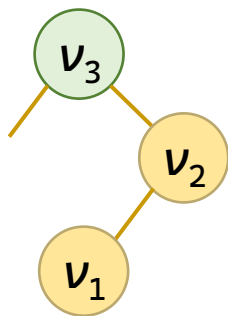
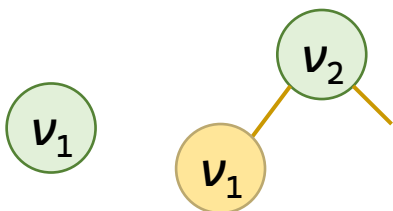
BSTNode* insertBST(BSTNode* root, int key) {
    // 递归出口：找到空位置，创建新节点并返回
    if (root == NULL)
        return createNode(key);
    // 如果关键字小于根节点，插入左子树
    if (key < root->data)
        root->lchild = insertBST(root->lchild, key);
    // 如果关键字大于根节点，插入右子树
    else if (key > root->data)
        root->rchild = insertBST(root->rchild, key);
    else
        printf("提示：关键字 %d 已存在于树中。\\n", key);
    return root;
}

```

二叉搜索树：创建

- 算法

- 遍历数组
- 插入元素



```
BSTNode* createBST(int arr[], int n) {  
    BSTNode* root = NULL;  
    for (int i = 0; i < n; i++) {  
        root = insertBST(root, arr[i]);  
    }  
    return root;  
}
```

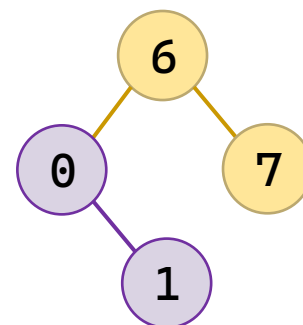
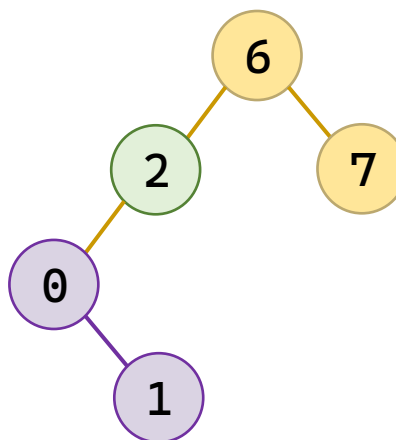
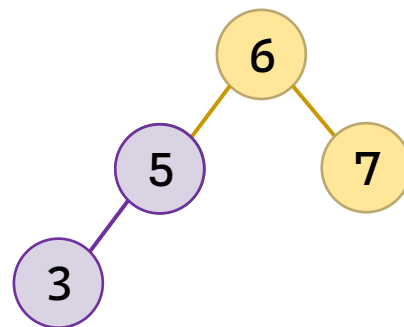
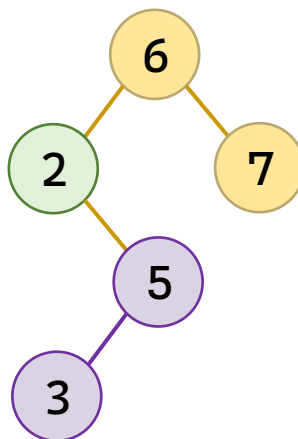
二叉搜索树：删除

- 算法

- 递归查找，找到元素

- 左子树为空，找右孩子

- 右子树为空，找左孩子

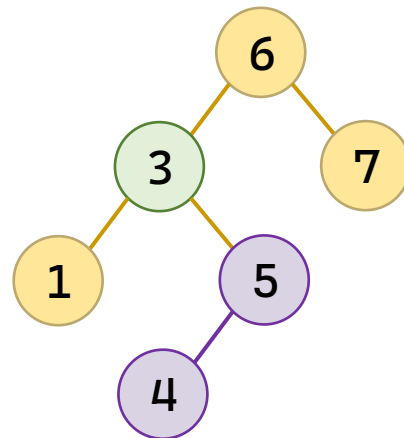
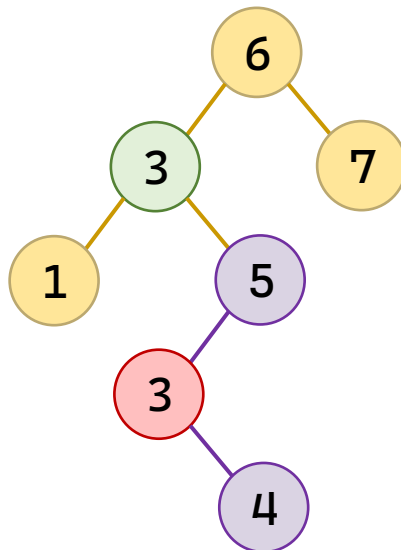
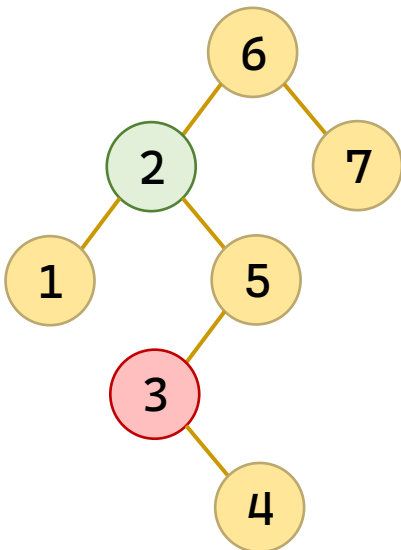


二叉搜索树：删除

- 算法

- 左右子树为不空

- 找到右子树最小值，即最往左的左子树
 - 替代当前值
 - 删除右子树里的最小值点



二叉搜索树：删除

- 算法

- 递归查找
找到元素

```
BSTNode* deleteBST(BSTNode* root, int key) {  
    if (key < root->data)  
        root->lchild = deleteBST(root->lchild, key);  
    else if (key > root->data)  
        root->rchild = deleteBST(root->rchild, key);
```

- 左子树为空，找右孩子

```
    if (root->lchild == NULL) {  
        BSTNode* temp = root->rchild;  
        free(root);  
        return temp;  
    }
```

- 右子树为空，找左孩子

```
    if (root->rchild == NULL) {  
        BSTNode* temp = root->lchild;  
        free(root);  
        return temp;  
    }
```

二叉搜索树：删除

- 算法

- 左右子树为不空

- 找到右子树最小值，即最往左的左子树

```
BSTNode* mostLeft = root->rchild;  
while (mostLeft->lchild != NULL) {  
    mostLeft = mostLeft->lchild;  
}
```

- 替代当前值

```
root->data = mostLeft->data;
```

- 删除右子树里的最小值点

```
root->rchild = deleteBST(root->rchild, mostLeft->data);
```

```

BSTNode* deleteBST(BSTNode* root, int key) {
    if (root == NULL) {
        printf("删除失败: 关键字 %d 不在树中.\n", key);
        return NULL;
    }
    // 第一步: 递归找到要删除的节点
    if (key < root->data) {
        root->lchild = deleteBST(root->lchild, key);
    } else if (key > root->data) {
        root->rchild = deleteBST(root->rchild, key);
    }
    else {        // 找到了! 开始处理删除逻辑
        if (root->lchild == NULL) {    // 情况 1: 左子树为空, 或叶子
            BSTNode* temp = root->rchild;
            free(root);                // 释放当前节点
            return temp;               // 让父节点直接指向右孩子
        }
        else if (root->rchild == NULL) { // 情况 2: 右子树为空
            BSTNode* temp = root->lchild;
            free(root);
            return temp;               // 让父节点直接指向左孩子
        }
    }
}

```



```

else {          // 情况 3: 左右子树都不为空 (最经典的情况)
    // 找到右子树中最小的节点 (即中序后继)
    BSTNode* mostLeft = root->rchild;
    while (mostLeft->lchild != NULL) {
        mostLeft = mostLeft->lchild;
    }
    // 用该最小节点的值覆盖当前要删除的节点
    root->data = mostLeft->data;
    // 在右子树中递归删除那个值相同的最小节点
    // (它一定没有左孩子, 退化为情况1)
    root->rchild = deleteBST(root->rchild, mostLeft->data);
}
}
return root;
}

```

二叉搜索树：查找

- 算法

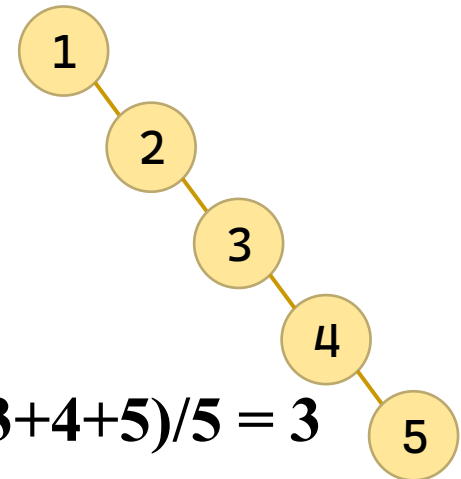
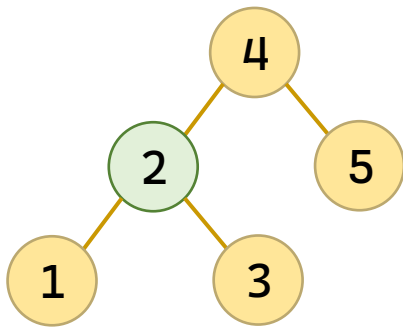
- 中序遍历算法

```
BSTNode* searchBST(BSTNode* root, int key) {  
    // 递归出口：要么找到空（没找到），要么找到了  
    if (root == NULL || root->data == key) {  
        return root;  
    }  
    // 根据大小关系决定去左子树还是右子树继续找  
    if (key < root->data) {  
        return searchBST(root->lchild, key);  
    } else {  
        return searchBST(root->rchild, key);  
    }  
}
```

二叉排序树-查找性能分析

- 查找性能分析

- 对于每一棵二叉排序树，均可按照平均查找长度的定义来计算它的ASL值。
- 由n个关键字值构造所得的不同形态的二叉排序树，其平均查找长度不一定相同，甚至可能差别很大。



$$ASL_1 = (3 + 2 + 3 + 1 + 2) / 5 = 2.2 \quad ASL_2 = (1 + 2 + 3 + 4 + 5) / 5 = 3$$

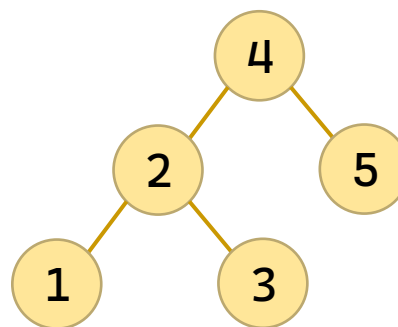
平衡二叉树

- 平衡二叉树 (Balanced Binary Tree)
 - 别称：AVL树，Adelson-Velskii and Landis Tree
- 定义
 - 或者是一棵空树，或者是具有下列性质的二叉树：
 - (1)左子树和右子树都是平衡二叉树；
 - (2)左子树和右子树的深度之差 ≤ 1 。

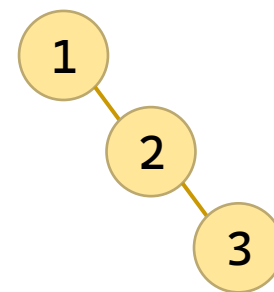
平衡二叉树：定义

- 平衡二叉树的存储

```
typedef struct AVLNode {  
    int data;                // 数据域  
    int height;              // 当前节点的高度  
    struct AVLNode* lchild, * rchild;  
} AVLNode;
```



平衡



不平衡

平衡二叉树：判断平衡

- 更新高度

- 后续遍历获得左右子树高度
- 当前高度是左右子树高度加1

```
int getHeight(AVLNode* node) {  
    return node ? node->height : 0;  
}  
void updateHeight(AVLNode* node) {  
    if (node) {  
        int leftH = getHeight(node->lchild);  
        int rightH = getHeight(node->rchild);  
        node->height = 1 +  
            (leftH > rightH ? leftH : rightH);  
    }  
}
```

- 平衡因子：左右子树高度差

```
int getBalanceFactor(AVLNode* node) {  
    return node ? getHeight(node->lchild) - getHeight(node->rchild) : 0;  
}
```

平衡二叉树：左旋转、右旋转

• 右旋转

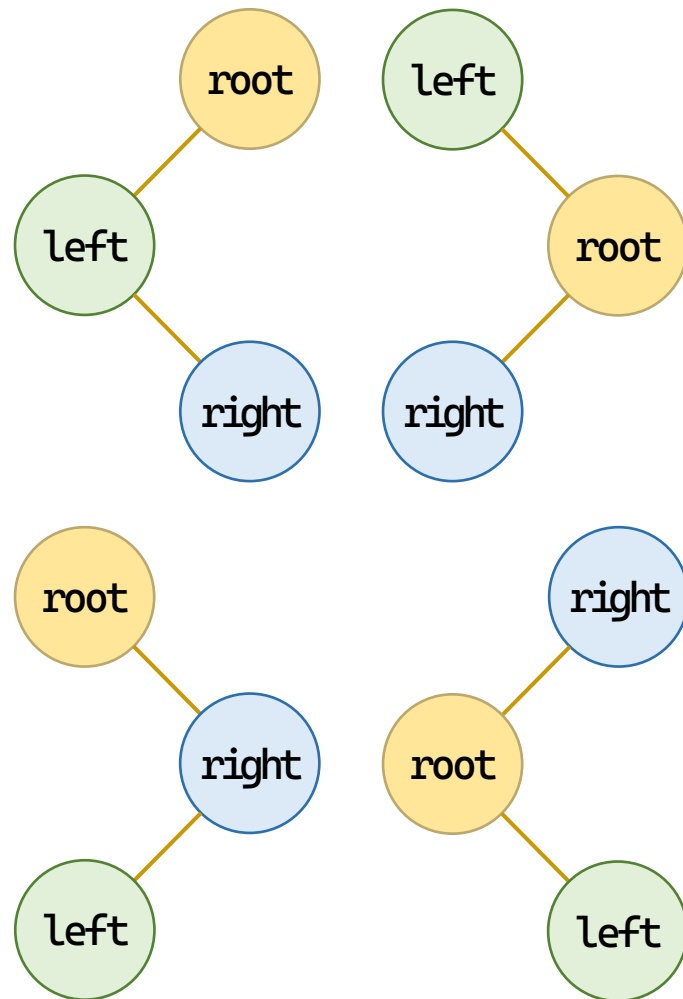
```
AVLNode* left = root->lchild;  
AVLNode* left_right = left->rchild;  
left->rchild = root;  
root->lchild = left_right;
```

• 左旋转

```
AVLNode* right = root->rchild;  
AVLNode* right_left = right->lchild;  
right->lchild = root;  
root->rchild = right_left;
```

• 旋转后更新涉及各树

```
updateHeight(root);  
updateHeight(left);
```



```

AVLNode* rightRotate(AVLNode* root) {
    AVLNode* left = root->lchild;
    AVLNode* left_right = left->rchild;
    left->rchild = root;
    root->lchild = left_right;
    updateHeight(root);
    updateHeight(left);
    return left;
}

```

```

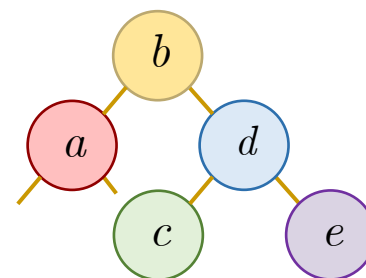
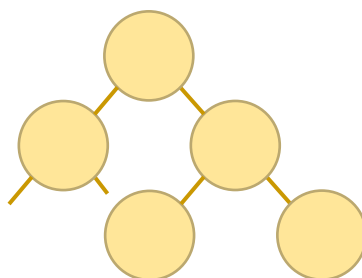
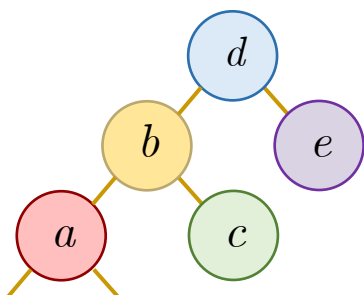
AVLNode* leftRotate(AVLNode* root) {
    AVLNode* right = root->rchild;
    AVLNode* right_left = right->lchild;
    right->lchild = root;
    root->rchild = right_left;
    updateHeight(root);
    updateHeight(right);
    return right;
}

```

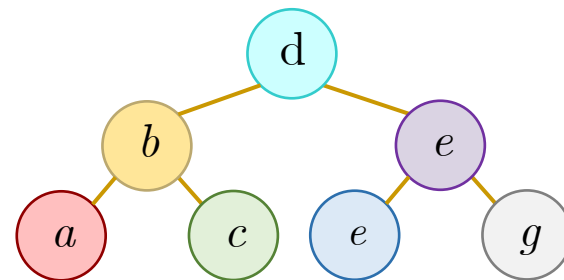
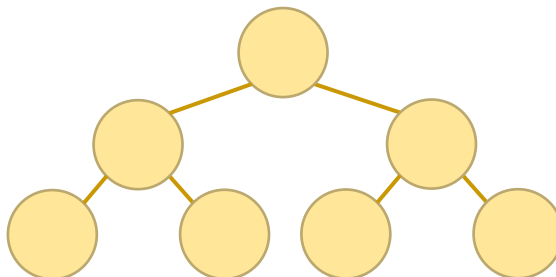
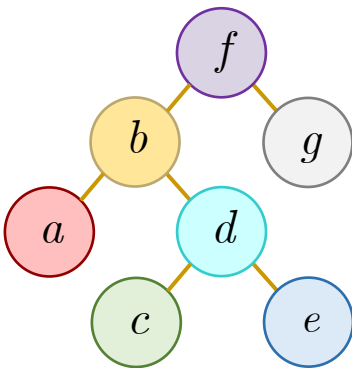

平衡二叉树：平衡

- 根结点左重（先画图，再填空）

- LL：左子树左重（或平衡）



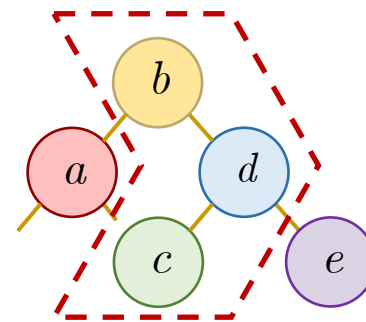
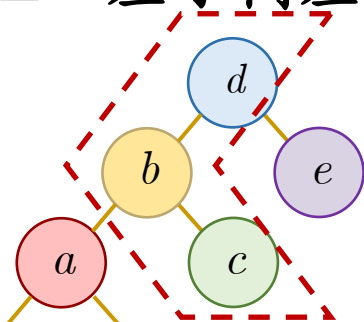
- LR：左子树右重



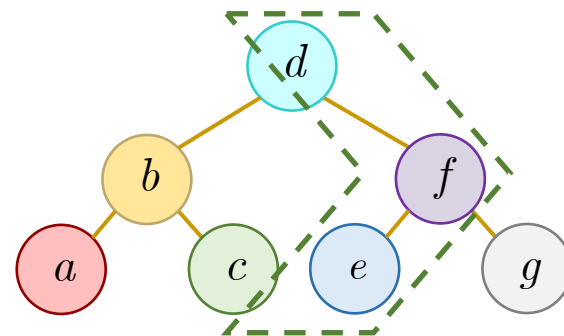
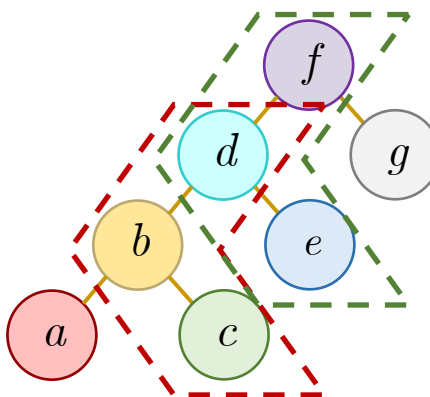
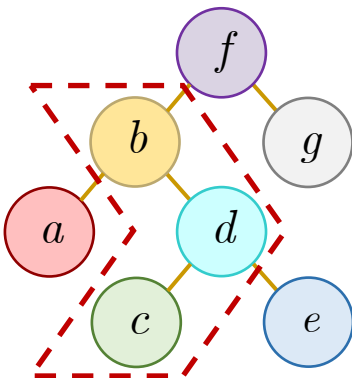
平衡二叉树：平衡

- 根结点左重

- LL：左子树左重（或平衡）



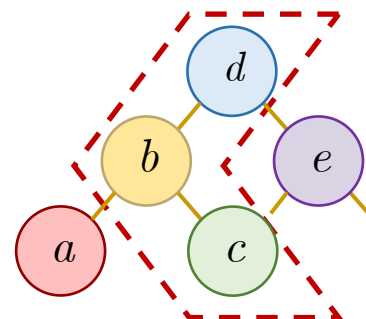
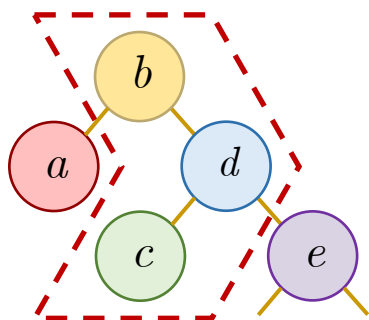
- LR：左子树右重



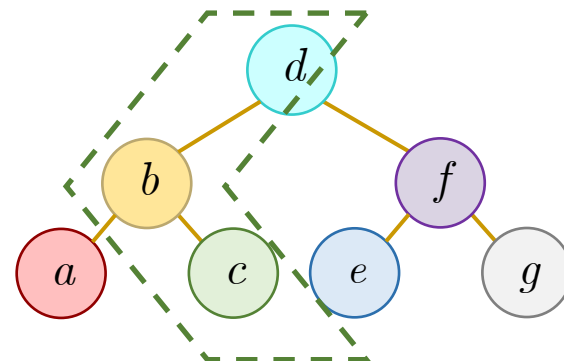
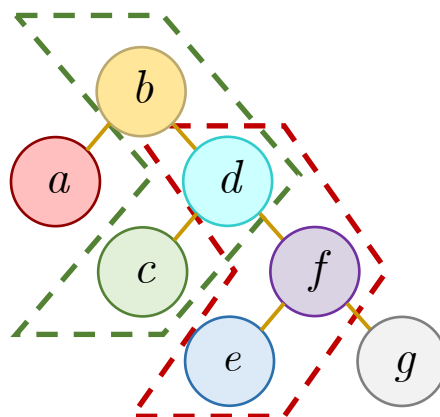
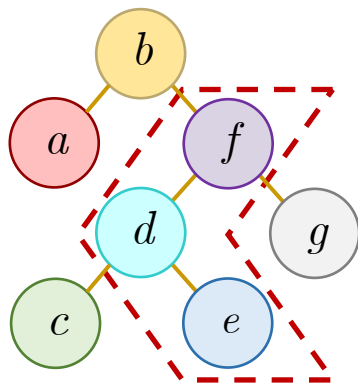
平衡二叉树：平衡

- 根结点右重（按根结点左重的情况镜像解决）

– RR：左子树右重（或平衡）



– RL：左子树左重



平衡二叉树：平衡

- 根结点左重

- LL：根结点右转

```
int bf = getBalanceFactor(node);
```

```
if (bf>1 && getBalanceFactor(node->lchild)>=0)
    return rightRotate(node);
```

- LR：左子树左转，根结点右转

```
if (bf > 1 && getBalanceFactor(node->lchild) < 0) {
    node->lchild = leftRotate(node->lchild);
    return rightRotate(node);
}
```

- 根结点右重

- RR：根结点左转

```
if (bf<-1 && getBalanceFactor(node->rchild)>=0)
    return leftRotate(node);
```

- RL：右子树右转，根结点左转

```
if (bf < -1 && getBalanceFactor(node->rchild) > 0) {
    node->rchild = rightRotate(node->rchild);
    return leftRotate(node);
}
```

```

AVLNode* balanceNode(AVLNode* node) {
    if (node == NULL) return NULL;
    int bf = getBalanceFactor(node);
    // LL 型: 左重且左孩子非右重 (>=0 表示左孩子本身左重或平衡)
    if (bf > 1 && getBalanceFactor(node->lchild) >= 0)
        return rightRotate(node);
    // LR 型: 左重且左孩子右重 (<0)
    if (bf > 1 && getBalanceFactor(node->lchild) < 0) {
        node->lchild = leftRotate(node->lchild);
        return rightRotate(node);
    }
    // RR 型: 右重且右孩子非左重 (<=0)
    if (bf < -1 && getBalanceFactor(node->rchild) <= 0)
        return leftRotate(node);
    // RL 型: 右重且右孩子左重 (>0)
    if (bf < -1 && getBalanceFactor(node->rchild) > 0) {
        node->rchild = rightRotate(node->rchild);
        return leftRotate(node);
    }
    return node;
}

```

平衡二叉树：插入

• 算法

– 后续遍历，找到插入位置

```
if (root == NULL)
    return createNode(key);
```

▪ 因为当前结点是否平衡取决于左右子树是否平衡

– 插入数据小于当前结点，在左子树插入

▪ 因为当前结点可能左右子树，不可能在当前结点插入

```
if (key < root->data)
    root->lchild = insertAVL(root->lchild, key);
```

– 插入数据大于当前结点，在右子树插入

```
else if (key > root->data)
    root->rchild = insertAVL(root->rchild, key);
```

– 插入后更新高度，重新平衡

```
updateHeight(root);
return balanceNode(root);
```

```

AVLNode* insertAVL(AVLNode* root, int key) {
    if (root == NULL) {
        return createNode(key);
    }

    if (key < root->data) {
        root->lchild = insertAVL(root->lchild, key);
    } else if (key > root->data) {
        root->rchild = insertAVL(root->rchild, key);
    } else {
        printf("提示: 关键字 %d 已存在, 插入忽略.\n", key);
        return root;
    }

    updateHeight(root);
    return balanceNode(root);
}

```

平衡二叉树：删除

• 算法

– 后续遍历，找到插入位置

```
if (root == NULL)
    return NULL;
```

– 删除数据小于当前结点，在左子树删除

- 因为当前结点可能左右子树，不可能在当前结点插入

```
if (key < root->data)
    root->lchild = deleteAVL(root->lchild, key);
```

– 删除数据大于当前结点，在右子树删除

```
else if (key > root->data)
    root->rchild = deleteAVL(root->rchild, key);
```

– 执行删除（见后页）

– 删除后更新高度，重新平衡

```
updateHeight(root);
return balanceNode(root);
```


平衡二叉树：删除

- 算法：执行删除（此时已找到）

- 左子结点空，
右子结点升级

```
if (root->lchild == NULL) {  
    AVLNode* temp = root->rchild;  
    free(root);  
    return temp;  
}
```

- 右子结点空，
左子结点升级

```
} else if (root->rchild == NULL) {  
    AVLNode* temp = root->lchild;  
    free(root);  
    return temp;  
}
```

- 左右结点都在，
用右节点的最小
结点顶替

```
} else {  
    AVLNode* temp = root->rchild;  
    while (temp->lchild != NULL)  
        temp = temp->lchild;  
    root->data = temp->data;  
    root->rchild = deleteAVL(  
        root->rchild, temp->data);  
}
```

```

AVLNode* deleteAVL(AVLNode* root, int key) {
    if (root == NULL) {
        printf("删除失败: 关键字 %d 不在树中.\n", key);
        return NULL;
    }
    if (key < root->data)
        root->lchild = deleteAVL(root->lchild, key);
    else if (key > root->data)
        root->rchild = deleteAVL(root->rchild, key);
    else { // 找到待删除节点
        if (root->lchild == NULL) {
            AVLNode* temp = root->rchild;
            free(root);
            return temp;
        } else if (root->rchild == NULL) {
            AVLNode* temp = root->lchild;
            free(root);
            return temp;
        } else {

```

```

// 双孩子：用右子树最小值（中序后继）顶替
AVLNode* temp = root->rchild;
while (temp->lchild != NULL)
    temp = temp->lchild;
root->data = temp->data;
root->rchild = deleteAVL(root->rchild, temp->data);
    }
}

if (root == NULL) return NULL;

updateHeight(root);
return balanceNode(root);
}

```

平衡二叉树：查找

- 算法

- 如相等，则停止
- 如过大，找左支
- 如过小，找右支

```
AVLNode* searchAVL(AVLNode* root, int key) {  
    if (root == NULL || root->data == key) {  
        return root;  
    }  
    if (key < root->data) {  
        return searchAVL(root->lchild, key);  
    } else {  
        return searchAVL(root->rchild, key);  
    }  
}
```

平衡二叉树：性能分析

- 平衡二叉树的查找性能分析：
 - 在平衡树上进行查找的过程和二叉排序树相同，因此，查找过程中和给定值进行关键字的比较次数不超过平衡树的深度。
- 在平衡二叉树上进行查找的时间复杂度为 $O(\log n)$
- 高度为 h 的 AVL 树所含的最少节点数 $N(h)$ 满足：
 - $$N(h) = N(h-1) + N(h-2) + 1$$

平衡二叉树：性能分析

- 问：含有 n 个关键字的平衡二叉树可能达到的最小和最大深度是多少？

$$h_{\min} = \lfloor \log_2 n \rfloor + 1, h_{\max} \leq 1.44 \log_2 (n + 2) - 0.328$$

$n=0$



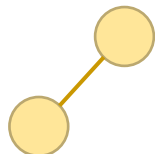
$h_{\max}=0$

$n=1$



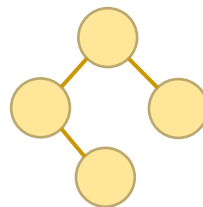
$h_{\max}=1$

$n=2$



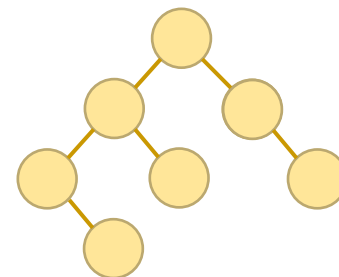
$h_{\max}=2$

$n=4$



$h_{\max}=3$

$n=7$



$h_{\max}=4$

B树和B⁺树

- 平衡二叉排序树便于动态查找，因此用平衡二叉排序树来组织索引表是一种可行的选择。当用于大型数据库时，所有数据及索引都存储在外存，因此，涉及到内、外存之间频繁的数据交换，这种交换速度的快慢成为制约动态查找的瓶颈。若以二叉树的结点作为内、外存之间数据交换单位，则查找给定关键字时对磁盘平均进行 $\log_2 n$ 次访问是不能容忍的，因此，必须选择一种能尽可能降低磁盘I/O次数的索引组织方式。树结点的大小尽可能地接近页的大小。

B树

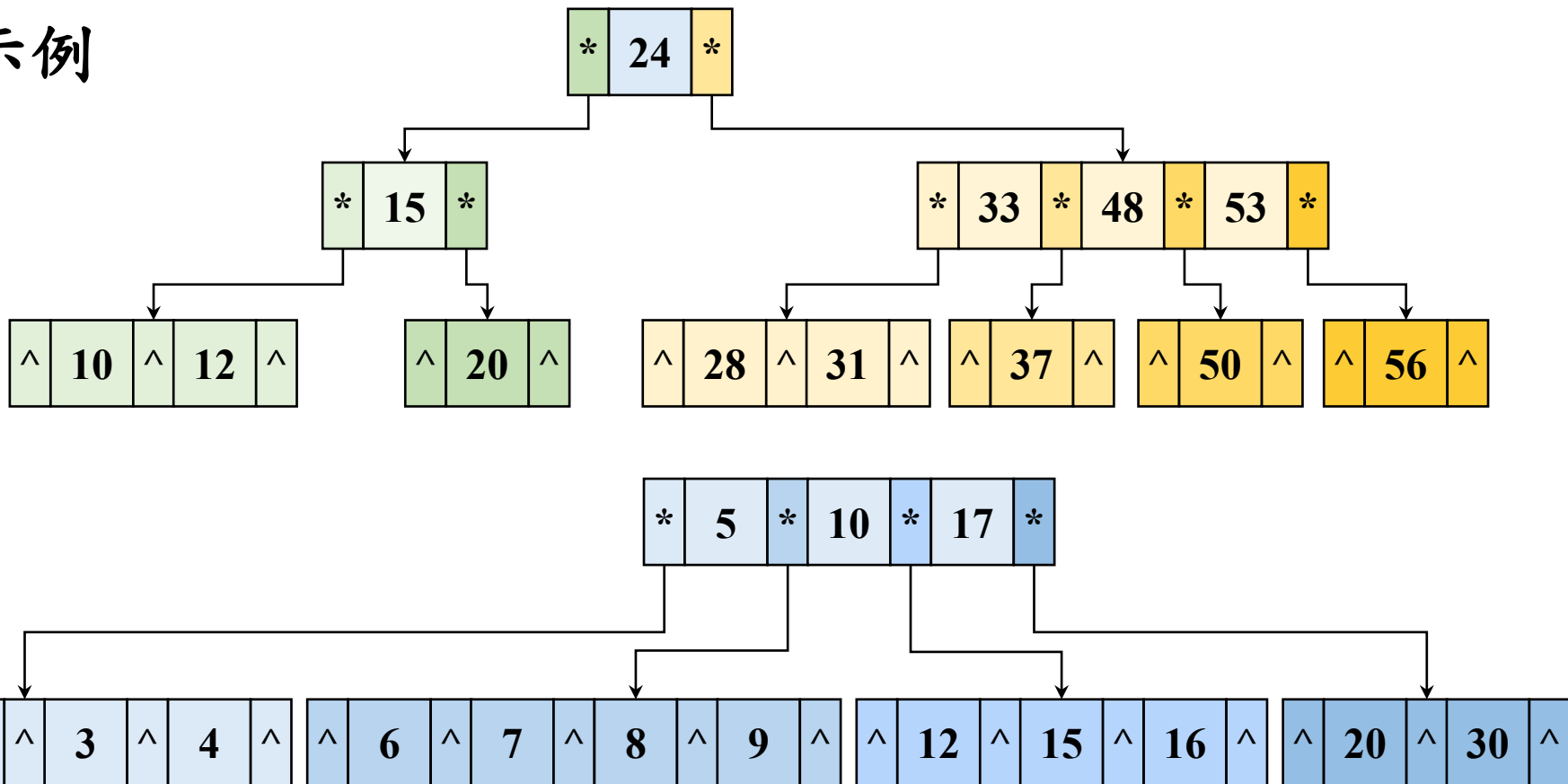
- R. Bayer和E. McCreight在1972年提出了一种多路平衡查找树，称为B树(其变型体是B+树)。
- 一个 m 阶的B树是一个有以下属性的树：
 - 每个节点最多有 m 个子节点。
 - 每个非叶子、非根节点至少有 $\lceil m/2 \rceil$ 个子节点。
 - 如果根节点不是叶子，则它至少有2个子节点。
 - 一个有 k 个子节点的非叶子节点，包含 $k-1$ 个键值。
 - 所有叶子节点都在同一层。

B树

- R. Bayer和E. McCreight在1972年提出了一种多路平衡查找树，称为B树(其变型体是B+树)。
- 一个 m 阶的B树是一个有以下属性的树：
 - 每个节点最多有 m 个子节点。
 - 每个非叶子、非根节点至少有 $\lceil m/2 \rceil$ 个子节点。
 - 如果根节点不是叶子，则它至少有2个子节点。
 - 一个有 k 个子节点的非叶子节点，包含 $k-1$ 个键值。
 - 所有叶子节点都在同一层。

B树

• 示例



B树

• B树的存储结构

```
#define T 3
#define MAX_KEYS (2 * T - 1)
#define MAX_CHILDREN (2 * T)

typedef struct BTreeNode {
    int keyCount;                /* 当前节点中关键字的数量 */
    int keys[MAX_KEYS];          /* 始终按升序存放关键字 */
    void *recptr[MAX_KEYS];      /* 与关键字对应的记录指针 */
    bool isLeaf;                 /* 叶子节点没有孩子 */
    struct BTreeNode *children[MAX_CHILDREN];
} BTreeNode;
```

B树：查找

- 算法

- 找到首次小于关键字的索引

```
while (index < node->keyCount && key > node->keys[index])  
    ++index;
```

- 找到就返回

```
if (index < node->keyCount && key == node->keys[index])  
    return node;
```

- 否则搜索下一级区间

- 记录叶子节点，提前停止

```
return node->isLeaf ? NULL : searchNode(node->children[index], key);
```

```

bool contains(const BTreeNode *root, int key) {
    return root != NULL
        && searchNode((BTreeNode *)root, key) != NULL;
}

BTreeNode *searchNode(BTreeNode *node, int key) {
    int index = 0;

    while (index < node->keyCount && key > node->keys[index]) {
        ++index;
    }

    if (index < node->keyCount && key == node->keys[index]) {
        return node;
    }

    return node->isLeaf ? NULL :
        searchNode(node->children[index], key);
}

```

B树：建立结点

- 算法

```
BTreeNode *createNode(bool isLeaf) {  
    BTreeNode *node = (BTreeNode *)calloc(1, sizeof(BTreeNode));  
    if (node == NULL) {  
        perror("无法分配 B 树节点");  
        exit(EXIT_FAILURE);  
    }  
    node->isLeaf = isLeaf;  
    return node;  
}
```

B树：插入

- 算法

- 如果已有关键字则报错
- 如果空树则建立根

- 如果根满则分裂根

- 执行非空树插入

```
if (contains(*root, key))  
    return false;
```

```
if (*root == NULL) {  
    *root = createNode(true);  
    (*root)->keys[0] = key;  
    (*root)->recptr[0] = NULL;  
    (*root)->keyCount = 1;  
    return true;  
}
```

```
if ((*root)->keyCount == MAX_KEYS) {  
    newRoot = createNode(false);  
    newRoot->children[0] = *root;  
    splitChild(newRoot, 0);  
    *root = newRoot;  
}
```

```
insertNonFull(*root, key);
```

```

bool insert(BTreeNode **root, int key) {
    BTreeNode *newRoot;
    if (contains(*root, key))
        return false;
    if (*root == NULL) {
        *root = createNode(true);
        (*root)->keys[0] = key;
        (*root)->recptr[0] = NULL;
        (*root)->keyCount = 1;
        return true;
    }
    if ((*root)->keyCount == MAX_KEYS) {
        newRoot = createNode(false);
        newRoot->children[0] = *root;
        splitChild(newRoot, 0);
        *root = newRoot;
    }
    insertNonFull(*root, key);
    return true;
}

```


B树：插入

- 非空树插入

- 如果在叶子层插入，像顺序表一样后移、插入

```
for (i = list->length  
- 1; i >= index; i--)
```

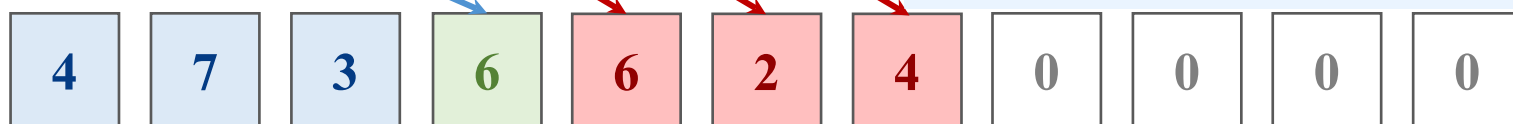
`list->length: 6`

```
list->length++;
```



`index: 3`

```
list->data[i+1] = list->data[i];
```



data



```
list->data[index] = data;
```

`SL_MAXSIZE`

- 如果非叶子节点，找到插入处，分裂满分支，再向下进入

B树：插入

- 非空树插入

- 如果在叶子层插入，

```
if (node->isLeaf) {
```

- 像顺序表
后移

```
while (index >= 0 && key < node->keys[index]) {  
    node->keys[index + 1] = node->keys[index];  
    node->recptr[index + 1] = node->recptr[index];  
    --index;  
}
```

- 插入

```
node->keys[index + 1] = key;  
node->recptr[index + 1] = NULL;  
++node->keyCount;  
return;
```

```
}
```

B树：插入

- 算法

- 如果非叶子节点，找到插入处

```
while (index >= 0 && key < node->keys[index])  
    --index;  
++index;
```

- 分裂满分支，避免回溯

```
if (node->children[index]->keyCount == MAX_KEYS) {  
    splitChild(node, index);  
    if (key > node->keys[index])  
        ++index;  
}
```

- 再向下进入

```
insertNonFull(node->children[index], key);
```

```

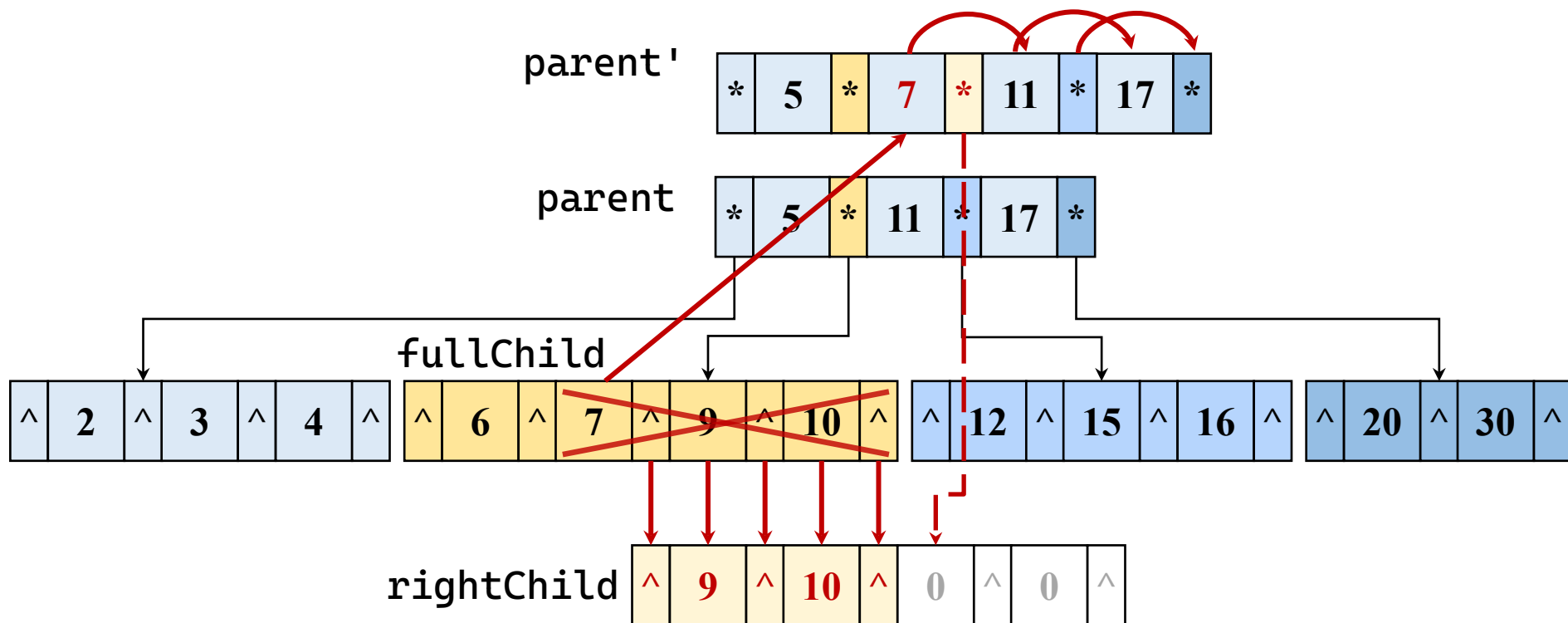
void insertNonFull(BTreeNode *node, int key) {
    int index = node->keyCount - 1;
    if (node->isLeaf) {
        while (index >= 0 && key < node->keys[index]) {
            node->keys[index + 1] = node->keys[index];
            node->recptr[index + 1] = node->recptr[index];
            --index;
        }
        node->keys[index + 1] = key;
        node->recptr[index + 1] = NULL;
        ++node->keyCount;
        return;
    }
    while (index >= 0 && key < node->keys[index])
        --index;
    ++index;
    /* 向下进入前先分裂满孩子，避免回溯。 */
    if (node->children[index]->keyCount == MAX_KEYS) {
        splitChild(node, index);
        if (key > node->keys[index])
            ++index;
    }
    insertNonFull(node->children[index], key);
}

```

B树：分裂

- 算法

- 建右支，满支挪一半，父节点后半部右移，中子提一级



B树：分裂

• 算法

– 建右支

```
BTreeNode *full = parent->children[index];  
BTreeNode *right = createNode(fullChild->isLeaf);
```

– 满支挪一半

▪ 设置长度

```
right->keyCount = T - 1;  
full->keyCount = T - 1;
```

▪ 挪动主键、记录

```
for (i = 0; i < T - 1; ++i) {  
    right->keys[i] = fullChild->keys[i + T];  
    right->recptr[i] = fullChild->recptr[i + T];  
}
```

▪ 非叶子挪子分支

```
if (!fullChild->isLeaf)  
    for (i = 0; i < T; ++i)  
        right->children[i] = full->children[i + T];
```

– 父节点后半部右移

– 中子提一级

B树：分裂

- 算法

- 父节点后半部右移

- 挪分支

```
for (i = parent->keyCount; i >= index + 1; --i)
    parent->children[i + 1] = parent->children[i];
parent->children[index + 1] = rightChild;
```

- 挪主键

```
for (i = parent->keyCount - 1; i >= index; --i) {
    parent->keys[i + 1] = parent->keys[i];
    parent->recptr[i + 1] = parent->recptr[i];
}
```

- 中子提一级

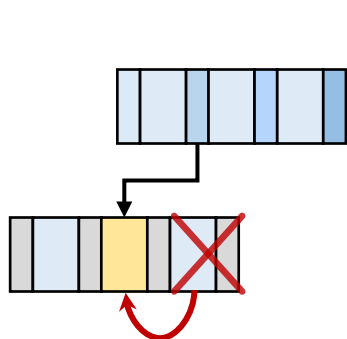
```
parent->keys[index] = full->keys[T - 1];
parent->recptr[index] = full->recptr[T - 1];
++parent->keyCount;
```

```

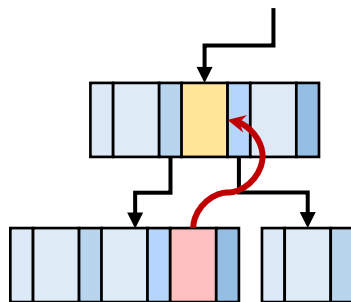
void splitChild(BTreeNode *parent, int index) {
    BTreeNode *fullChild = parent->children[index];
    BTreeNode *rightChild = createNode(fullChild->isLeaf);
    int i;
    rightChild->keyCount = T - 1;
    for (i = 0; i < T - 1; ++i) {
        rightChild->keys[i] = fullChild->keys[i + T];
        rightChild->recptr[i] = fullChild->recptr[i + T];
    }
    if (!fullChild->isLeaf)
        for (i = 0; i < T; ++i)
            rightChild->children[i] = fullChild->children[i+T];
    fullChild->keyCount = T - 1;
    for (i = parent->keyCount; i >= index + 1; --i)
        parent->children[i + 1] = parent->children[i];
    parent->children[index + 1] = rightChild;
    for (i = parent->keyCount - 1; i >= index; --i) {
        parent->keys[i + 1] = parent->keys[i];
        parent->recptr[i + 1] = parent->recptr[i];
    }
    parent->keys[index] = fullChild->keys[T - 1];
    parent->recptr[index] = fullChild->recptr[T - 1];
    ++parent->keyCount;
}

```

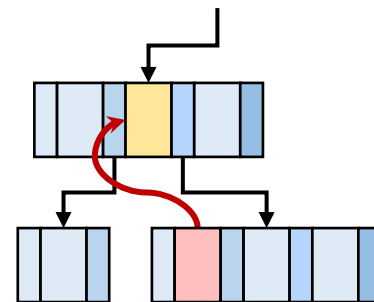

B树：删除



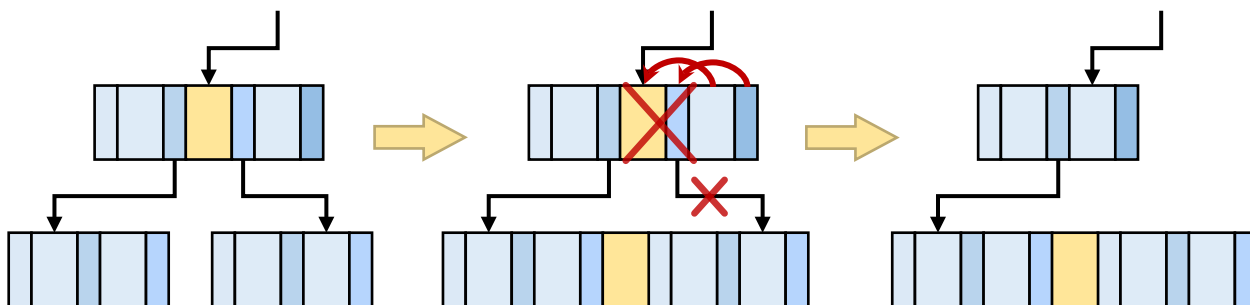
A1：命中叶子
直接删除



A2：命中非叶，左富右穷
左支幼子上移
(移：删除原节点)

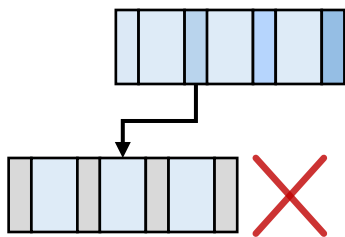


A3：命中非叶，左穷右富
右支长子上移

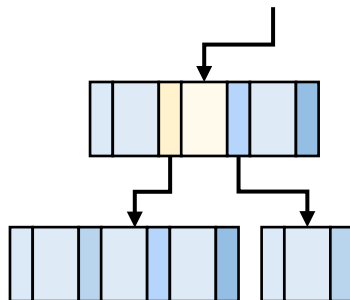


A4：命中非叶，左右都穷，左右合并
合并：主键下沉为中子，右支复制到左支

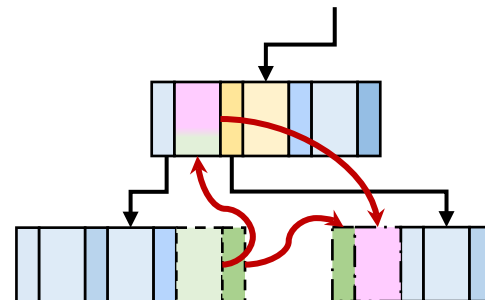
B树：删除



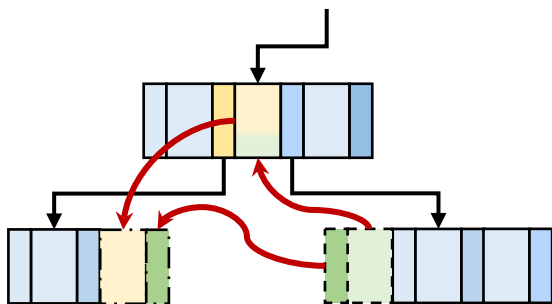
B1：未命中，停在叶子
直接报错



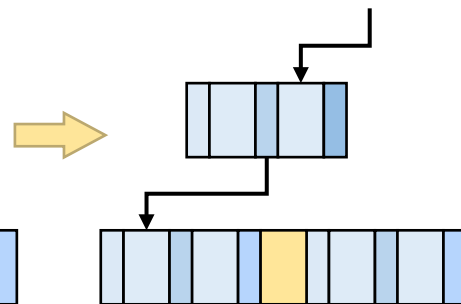
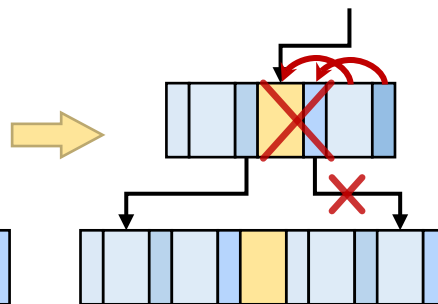
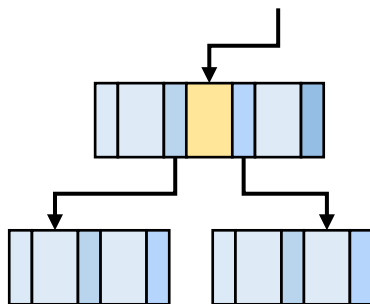
B2：未命中，非叶
进入左支继续删



B21：删左支前左兄富
借左兄的幼子



B22：删左支前左兄富
借左兄的幼子



B23：删左支前左右穷

合并：非最后一支：主键下沉为中子，右支复制到本支
最后一支：主键左兄下沉为中子，本支复制到左支

B树：删除

- 算法

- 找到位置：第一个大于等于key的位置

```
int index = 0;  
while (index < node->keyCount && key > node->keys[index])  
    ++index;
```

- A：如果命中

```
if (index < node->keyCount && node->keys[index] == key) {
```

- A1：叶子，if (node->isLeaf) {

直接删除

```
while (index < node->keyCount - 1) {  
    node->keys[index] = node->keys[index + 1];  
    node->recptr[index] = node->recptr[index + 1];  
    ++index;  
}  
--node->keyCount;
```

```
}
```

B树：删除

• 算法

– A2：命中非叶，左富右穷，左支幼子上移，删除原节点

```
if (node->children[index]->keyCount >= T) {  
    int replacement = predecessor(node->children[index]);  
    node->keys[index] = replacement;           // 替换当前关键字  
    removeFromNode(node->children[index], replacement); // 递归删除前驱
```

– A3：命中非叶，左穷右富，右支长子上移

```
} else if (node->children[index + 1]->keyCount >= T) {  
    int replacement = successor(node->children[index + 1]);  
    node->keys[index] = replacement;  
    removeFromNode(node->children[index + 1], replacement);
```

– A4：命中非叶，左右都穷，左右合并

```
else {  
    mergeChildren(node, index);  
    removeFromNode(node->children[index], key);  
}
```

B树：删除

• 算法

– B1：未命中，停在叶子，直接报错

```
if (node->isLeaf)
    return;
```

– B2：未命中，非叶，进入左支继续删

```
...
removeFromNode(node->children[index], key);
```

▪ 删之前要保证孩子至少有T个关键字

– B21：删左支前左兄富，借左兄的幼子

```
if (node->children[index]->keyCount == T - 1) {
    if (index > 0 && node->children[index - 1]->keyCount >= T)
        borrowFromPrevious(node, index);    // 从左兄弟借
```

– B22：删左支前左兄富，借左兄的幼子

```
else if (index < node->keyCount
        && node->children[index + 1]->keyCount >= T)
    borrowFromNext(node, index);    // 从右兄弟借
```

B树：删除

- 算法

- B23：删左支前左右穷

- 合并：非最后一支：主键下沉为中子，右支复制到本支

```
else if (index < node->keyCount)
    mergeChildren(node, index);
```

- 合并：最后一支：主键左兄下沉为中子，本支复制到左支

```
else {
    mergeChildren(node, index - 1);
    --index;    // 合并后，要调整索引
}
```

```

void removeFromNode(BTreeNode* node, int key) {
    int index = 0;
    /* 在当前节点中查找 key 的位置：找到第一个 >= key 的关键字位置 */
    while (index < node->keyCount && key > node->keys[index])
        ++index;
    /* ===== 情况 1: 在当前节点找到了 key ===== */
    if (index < node->keyCount && node->keys[index] == key) {
        /* 1a. 叶子节点：直接删除 */
        if (node->isLeaf) {
            /* 将 index 后面的关键字前移覆盖 */
            while (index < node->keyCount - 1) {
                node->keys[index] = node->keys[index + 1];
                node->recptr[index] = node->recptr[index + 1];
                ++index;
            }
            --node->keyCount;
        }
        /* 1b. 内部节点：用前驱或后继替换 */
        else {
            /* 如果左孩子有富余 (>= T 个关键字)，用前驱替换 */
            if (node->children[index]->keyCount >= T) {

```

```

        int replacement = predecessor(node->children[index]);
        node->keys[index] = replacement; // 替换当前关键字
        removeFromNode(node->children[index],
                        replacement); // 递归删除前驱
    }
    /* 否则如果右孩子有富余, 用后继替换 */
    else if (node->children[index+1]->keyCount >= T) {
        int replacement =
            successor(node->children[index + 1]);
        node->keys[index] = replacement;
        removeFromNode(node->children[index + 1],
                        replacement);
    }
    /* 两个孩子都只有 T-1 个关键字, 则合并, 然后递归删除 */
    else {
        mergeChildren(node, index);
        removeFromNode(node->children[index], key);
    }
}
return; // 删除完成, 返回
}

```



```

/* ===== 情况 2: 未在当前节点找到 key ===== */
/* 如果是叶子节点, 说明树中不存在该 key */
if (node->isLeaf)
    return;
/* 否则进入孩子继续删除, 但先要保证该孩子至少有 T 个关键字 */
if (node->children[index]->keyCount == T - 1) {
    // 优先考虑从兄弟借一个关键字 (左兄弟或右兄弟)
    // 借位条件: 兄弟的关键字数 >= T (即有富余)
    if (index > 0 && node->children[index-1]->keyCount >= T)
        borrowFromPrevious(node, index);    // 从左兄弟借
    else if (index < node->keyCount
             && node->children[index + 1]->keyCount >= T)
        borrowFromNext(node, index);        // 从右兄弟借
    // 如果无法借位 (兄弟也都只有 T-1 个关键字), 则合并
    else if (index < node->keyCount)
        // 如果 index 在 keyCount 范围内, 则当前孩子有右兄弟,
        // 将当前孩子与右兄弟合并 (合并时父节点关键字 index 下移)
        mergeChildren(node, index);
}

```

```

else {
    // 如果 index == keyCount, 说明当前孩子是最右边的孩子,
    // 只能与左兄弟合并 (合并时父节点关键字 index-1 下移)
    mergeChildren(node, index - 1);
    --index;
    // 合并后, 原来 index 位置的孩子现在是合并后的左孩子
    // 所以递归时要调整索引
}
}

/* 递归进入孩子继续删除 */
removeFromNode(node->children[index], key);
}

```

B树：删除相关：前驱和后继

- 前驱是子树的最大关键字，也是最幼子孙中的最大值

```
int predecessor(const BTreeNode *node) {  
    while (!node->isLeaf)  
        node = node->children[node->keyCount];  
    return node->keys[node->keyCount - 1];  
}
```

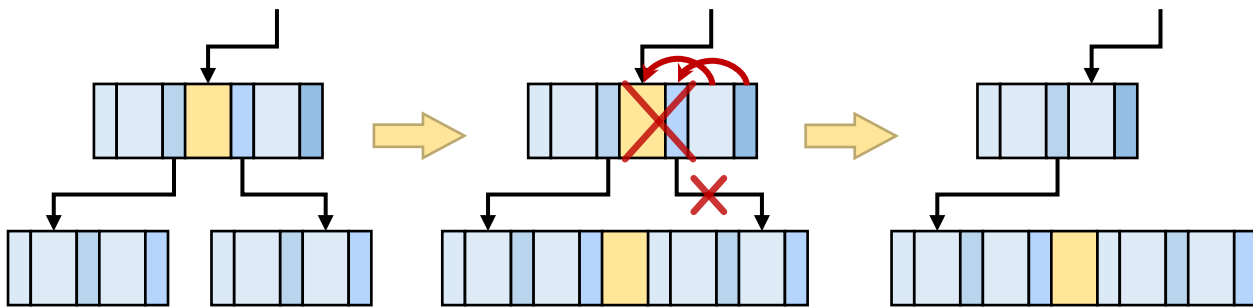
- 后继是子树的最小关键字，也是最长子孙中的最小值

```
int successor(const BTreeNode *node) {  
    while (!node->isLeaf)  
        node = node->children[0];  
    return node->keys[0];  
}
```

B树：删除相关：合并

• 算法

- 将父节点关键字下移到左子末尾



```
left->keys[leftCount] = parent->keys[index];  
left->recptr[leftCount] = parent->recptr[index];
```

- 右孩子追加到左孩子后

```
for (i = 0; i < right->keyCount; ++i) {  
    left->keys[leftCount + 1 + i] = right->keys[i];  
    left->recptr[leftCount + 1 + i] = right->recptr[i];  
}
```

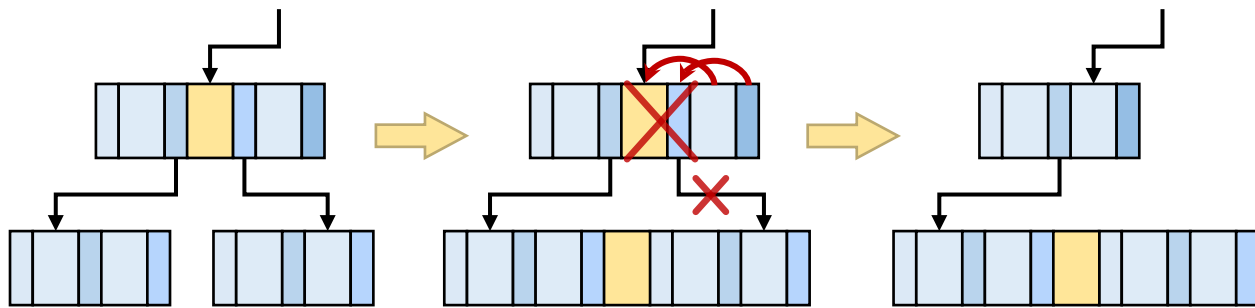
- 如果非叶子，还要复制指针

```
if (!left->isLeaf)  
    for (i = 0; i <= right->keyCount; ++i)  
        left->children[leftCount + 1 + i] = right->children[i];  
left->keyCount += right->keyCount + 1;
```

B树：删除相关：合并

- 算法

- 删除父节点
关键字



```
for (i = index; i < parent->keyCount - 1; ++i) {  
    parent->keys[i] = parent->keys[i + 1];  
    parent->recptr[i] = parent->recptr[i + 1];  
    parent->children[i + 1] = parent->children[i + 2];  
}  
--parent->keyCount;
```

- 释放右孩子

```
free(right);
```

```

void mergeChildren(BTreeNode* parent, int index) {
    BTreeNode* left = parent->children[index];      // 左孩子
    BTreeNode* right = parent->children[index + 1]; // 右孩子
    int leftCount = left->keyCount;    // 记录左孩子原有关键字数
    int i;
    // 第一步：将父节点的分隔关键字下移到左孩子的末尾
    left->keys[leftCount] = parent->keys[index];
    left->recptr[leftCount] = parent->recptr[index];
    // 第二步：将右孩子的所有关键字和记录指针追加到左孩子后面
    for (i = 0; i < right->keyCount; ++i) {
        left->keys[leftCount + 1 + i] = right->keys[i];
        left->recptr[leftCount + 1 + i] = right->recptr[i];
    }
    // 第三步：如果是内部节点，还要将右孩子的孩子指针追加到左孩子后面
    if (!left->isLeaf)
        for (i = 0; i <= right->keyCount; ++i)
            left->children[leftCount + 1 + i]
                = right->children[i];
    // 左孩子的关键字总数：原有 + 1（下移的父关键字） + 右孩子的全部
    left->keyCount += right->keyCount + 1;
}

```

```

// 第四步：从父节点中删除索引为 index 的关键字以及右孩子指针
// 将父节点中 index 后面的所有关键字和孩子指针依次前移一位
for (i = index; i < parent->keyCount - 1; ++i) {
    parent->keys[i] = parent->keys[i + 1];
    parent->recptr[i] = parent->recptr[i + 1];
    parent->children[i + 1] = parent->children[i + 2];
}
--parent->keyCount;          // 父节点关键字数减 1

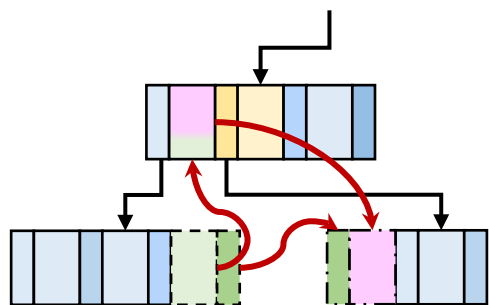
/* 释放右孩子节点 */
free(right);
}

```

B树：删除相关：借子

• 向左兄借子的算法

– 本人腾位置



– 接父关键字

– 迎接左兄幼子

– 左兄关键字上提

```
for (i = child->keyCount - 1; i >= 0; --i) {
    child->keys[i + 1] = child->keys[i];
    child->recptr[i + 1] = child->recptr[i];
}
if (!child->isLeaf)
    for (i = child->keyCount; i >= 0; --i) {
        child->children[i + 1] = child->children[i];
    }
```

```
child->keys[0] = parent->keys[index - 1];
child->recptr[0] = parent->recptr[index - 1];
```

```
if (!child->isLeaf)
    child->children[0]
        = sibling->children[sibling->keyCount];
```

```
parent->keys[index - 1] = sibling->keys[sibling->keyCount - 1];
parent->recptr[index - 1] = sibling->recptr[sibling->keyCount - 1];
```


B树：删除相关：借子

• 向右兄借子的算法

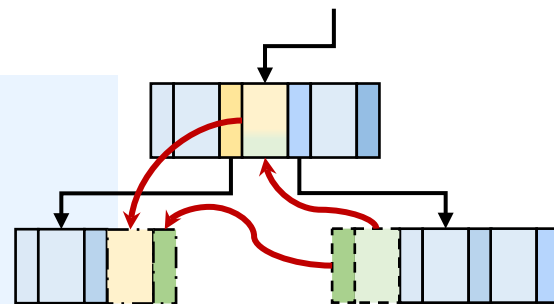
– 接父关键字 `child->keys[child->keyCount] = parent->keys[index];`
`child->recptr[child->keyCount] = parent->recptr[index];`

– 迎接右兄长子 `if (!child->isLeaf)`
`child->children[child->keyCount + 1]`
`= sibling->children[0];`

– 右兄关键字上提 `parent->keys[index] = sibling->keys[0];`
`parent->recptr[index] = sibling->recptr[0];`

– 右兄剩余孩子补缺

```
for (i = 1; i < sibling->keyCount; ++i) {  
    sibling->keys[i - 1] = sibling->keys[i];  
    sibling->recptr[i - 1] = sibling->recptr[i];  
}  
if (!sibling->isLeaf)  
    for (i = 1; i <= sibling->keyCount; ++i)  
        sibling->children[i-1] = sibling->children[i];
```



```

void borrowFromPrevious(BTreeNode* parent, int index) {
    BTreeNode* child = parent->children[index];      // 当前孩子
    BTreeNode* sibling = parent->children[index - 1]; // 左兄弟
    int i;
    // 第一步：为当前孩子腾出第一个位置（所有关键字和记录后移一位）
    for (i = child->keyCount - 1; i >= 0; --i) {
        child->keys[i + 1] = child->keys[i];
        child->recptr[i + 1] = child->recptr[i];
    }
    // 如果是内部节点，孩子指针也要后移（为从兄弟过来的孩子指针腾位置）
    if (!child->isLeaf)
        for (i = child->keyCount; i >= 0; --i)
            child->children[i + 1] = child->children[i];
    // 第二步：将父节点中位于index-1的关键字下移到当前孩子的第一个位置
    child->keys[0] = parent->keys[index - 1];
    child->recptr[0] = parent->recptr[index - 1];
    // 第三步：将左兄弟的最后一个孩子过继给当前孩子作为第一个孩子
    // 因为左兄弟的关键字要上提，它的最后一个孩子就一起移动到当前孩子。
    if (!child->isLeaf)
        child->children[0] = sibling->children[sibling->keyCount];
}

```

```

// 第四步：将左兄弟的最后一个关键字上提到父节点，替代原来的分隔关键字
parent->keys[index-1] = sibling->keys[sibling->keyCount - 1];
parent->recptr[index-1] = sibling->recptr[sibling->keyCount-1];
/* 左兄弟关键字数减 1，当前孩子关键字数加 1 */
--sibling->keyCount;
++child->keyCount;
}

void borrowFromNext(BTreeNode* parent, int index) {
    BTreeNode* child = parent->children[index];          // 当前孩子
    BTreeNode* sibling = parent->children[index + 1];      // 右兄弟
    int i;
    // 第一步：将父节点的分隔关键字下移到当前孩子的末尾
    child->keys[child->keyCount] = parent->keys[index];
    child->recptr[child->keyCount] = parent->recptr[index];
    // 第二步：将右兄弟的第一个孩子过继给当前孩子作为最后一个孩子
    // 因为右兄弟的第一个关键字要上提，它的第一个孩子应该移到当前孩子。
    if (!child->isLeaf)
        child->children[child->keyCount+1] = sibling->children[0];
    // 第三步：将右兄弟的第一个关键字上提到父节点，替代原来的分隔关键字
    parent->keys[index] = sibling->keys[0];
    parent->recptr[index] = sibling->recptr[0];
}

```

// 第四步：将右兄弟中剩余的关键字和孩子指针整体前移一位（填补空缺）

```
for (i = 1; i < sibling->keyCount; ++i) {
    sibling->keys[i - 1] = sibling->keys[i];
    sibling->recptr[i - 1] = sibling->recptr[i];
}
if (!sibling->isLeaf) {
    for (i = 1; i <= sibling->keyCount; ++i) {
        sibling->children[i - 1] = sibling->children[i];
    }
}
```

/* 更新关键字数：右兄弟减少，当前孩子增加 */

```
--sibling->keyCount;
++child->keyCount;
```

```
}
```

B⁺树

- 在实际文件系统中，使用B树的变体，称为m阶B^s树。
 - 它与B树的主要不同是：只在叶子结点中存储记录。
 - 在B⁺树中，所有的非叶子结点是索引，关键字是分界，用来界定某一关键字的记录所在的子树。
- 一棵m阶B⁺树与m阶B树的主要差异是：
 - 若一个结点有n棵子树，则必含有n个关键字；
 - 所有叶子结点中包含了全部记录的关键字信息及其记录的指针，而且叶子结点按关键字的大小从小到大顺序链接；
 - 所有的非叶子结点可以看成是索引的部分，结点中只含有其子树的根结点中的最大(或最小)关键字。

B⁺树

• B树的存储结构

```
#define ORDER 5
typedef struct BPlusNode {
    bool isLeaf;
    int keyCount;
    int keys[ORDER];           // 多留一个位置，供分裂前的暂存溢出使用
    void* recptr[ORDER];       // 仅叶子节点使用，与 keys 一一对应
    struct BPlusNode* children[ORDER + 1]; // 最多 ORDER+1 个孩子
    struct BPlusNode* parent;
    struct BPlusNode* next;    // 仅叶子节点使用，连接下一片叶子
} BPlusNode;
```

B⁺树

• B树的存储结构（联合体版）

```
#define ORDER 5
typedef struct BPlusNode {
    bool isLeaf;
    int keyCount;
    int keys[ORDER]; // 两者共用
    union {
        struct { // 叶子节点专用的部分
            void* recptr[ORDER];
            struct BPlusNode* next;
        } leaf;
        struct { // 内部节点专用的部分
            struct BPlusNode* children[ORDER + 1];
        } internal;
    } u;
    struct BPlusNode* parent; // 两者共用，放在外面简化访问
} BPlusNode;
```

B+树

- 在B+树上进行随机查找、插入、删除的过程基本上和B_树类似。
- B+树查找：
 - 与B_树相比，对B+树不仅可以从根结点开始按关键字随机查找，而且可以从最小关键字起，按叶子结点的链接顺序进行顺序查找。
 - 在B+树上进行随机查找时，若非叶子结点的关键字等于给定的K值，并不终止，而是继续向下直到叶子结点(只有叶子结点才存储记录)，即无论查找成功与否，都走了一条从根结点到叶子结点的路径。

B⁺树

- B⁺树插入：

- B⁺树的插入仅仅在叶子结点上进行。当叶子结点中的关键字个数大于 m 时，分裂为两个结点，含关键字个数分别是 $\lfloor (m+1)/2 \rfloor$ 和 $\lceil (m+1)/2 \rceil$ ，且将其中各自的最大关键字提升到父结点中，替代原结点在父结点中对应的关键字。提升后父结点又可能会分裂，依次类推。

- B⁺树删除：

- B⁺树的删除也仅在叶子结点进行，当叶子结点中的最大关键字被删除时，其在非叶子结点中的值可以作为一个“分界关键字”存在。若因删除而使结点中关键字的个数少于 $\lceil m/2 \rceil$ 时，其和兄弟结点的合并过程也和B树类似。

本节小结

- 介绍二叉排序树的基本概念、查找算法、插入算法和删除算法。
- 依据数据元素构造二叉排序树，其形状跟数据及其输入顺序都有关。
- 中序遍历二叉排序树可以得到排序序列。
- 介绍8种二叉排序树的基本平衡方法。
- 介绍B树和B⁺树的概念，查找、插入和删除操作等。

目录

	1	基本概念和分类
	2	静态查找
	3	动态查找
	4	哈希表

哈希表的概念

- 哈希（ Hashing， 又称：散列）
 - 一种将任意长度的输入，通过一个确定的函数，映射为固定长度的输出的过程。
 - 输入，称为关键字 $k \in K$ ；输出，称为哈希值 $h(k)$ 。
 - 哈希函数为 $h: K \rightarrow A$ ，其中：
 - K 为关键字全域（可能为无限集或极大有限集），
 $A = \{0, 1, \dots, m-1\}$ 为一个有限地址空间（长度为 m ）。
 - 一般 $|K| \gg m$ （或 $|K| > m$ ）时，该映射必定不是单射。
- 哈希表（ Hash List，散列表）是计算直达的存储结构。
 - 基于哈希函数建立的顺序表。

哈希表的概念

- 哈希的本质是压缩映射。
- 冲突 (Collision)
 - 对于 $k_1 \neq k_2$ ，若 $h(k_1) = h(k_2)$ ，则称 k_1, k_2 为一对同义词。
 - 根据鸽巢原理，冲突在有限地址空间中不可避免。
- 装填因子 α (关键性能指标) :
 - 定义： $\alpha = n/m$ ，其中 n 为表中已存储的记录个数， m 为哈希表的总长度。
 - α 越小，空间冗余度越大，时间效率越高。

哈希的构造方法

• 1、直接定址法

- 哈希函数： $H(k)=a \cdot k+b$
- 适合于：关键字分布连续且密集的场景
- 例：统计某班学生年龄，学号20262201—20262250，可令 $a=1, b=0$ ，直接保存。

• 2、数字分析法；

- 不直接拿关键字算，而是分析关键字的组成结构，提取出“随机性最强”的部分做地址。
- 例：上题，取 $a=1, b=-20262200$ 。
- 适合于：静态数据集，关键字有结构。

哈希的构造方法

• 3、平方取中法

- 方法：关键字平方（自己乘以自己），取中间几位。
 - 中间几位的值受 Key 所有位的影响（高位和低位都会乘进去），比直接取后几位更随机。
- 例如： $k=1234$ ，取万、千位，得到22（15**22**756）

• 4、折叠法

- 方法：把长数字切碎，再叠起来求和。
- 适用于：关键字位数特别长的情况，如：身份证号。
 - 移位折叠：123456789，按 $123+456+789=1368$ ，取368
 - 边界折叠：123456789，按 $123+654+789=1566$ ，取566

哈希的构造方法

• 5、除留余数法

– 方法： $H(k) = \text{mod}(k, p)$, $p \leq m$ ，建议 p 为质数。

- 因为取模运算的本质是“去掉了 Key 中能被 p 整除的公因子”，素数能最大程度保留余数的随机性，避开底层数据可能存在的周期性规律（比如全是偶数或全是 5 的倍数）。

– 例如： $p=7$ 时， $10\%7=3$ ， $20\%7=6$ ， $30\%7=2\ldots\ldots$

• 6、随机数法

– 方法：`srand(key); return rand()%m;`

– 缺点：方法慢，极少单独用于工程。

哈希的构造方法

- 选取哈希函数，考虑以下因素：
 - 计算哈希函数所需时间；
 - 关键字的长度；
 - 哈希表长度（哈希地址范围）；
 - 关键字分布情况；
 - 记录的查找频率。

处理冲突的方法

- 处理冲突：为产生冲突的哈希地址寻找新哈希地址。
- 1. 开放定址法
 - 为产生冲突的哈希地址 $H(k)$ 求得一个地址序列： $H_0, H_1, \dots, H_s, 1 \leq s \leq m-1$ ，其中， $H_0 = H(k)$ ， $H_i = \text{mod}(H_0 + d_i, m)$ ， $1 \leq i \leq s$ ， d_i 称为增量
 - (1) 线性探测再散列： $d_i = c \cdot i$ ，其中 c 为常数
 - (2) 二次探测再散列： $d_i = c_1 \cdot i^2 + c_2 \cdot i$
 - (3) 双重哈希再散列： $d_i = i \cdot h_2(k)$

处理冲突的方法

例 设定哈希函数 $H(\text{key}) = \text{key} \% 11$ ，采用线性探测再散列处理： $d_i = i$ ，则

关键字值	19	01	23	14	55	68	11	82	36	
地址初值	8	1	1	3	0	2	0	5	3	
哈希地址	8	1	2	3	0	4	5	6	7	

$$\begin{aligned} \rightarrow \text{ASL} &= (1 + 1 + 2 + 1 + 1 + 3 + 6 + 2 + 5) / n \\ &= 22/9 \end{aligned}$$

处理冲突的方法

• 2. 链地址法

- 将所有哈希地址相同的数据元素都存放在同一个链表中；
- 链地址法的平均查找长度较短；
- 链地址法更适合于：构造表前无法确定表长的情况(动态申请链结点)；
- 在用链地址法构造的哈希表中，删除结点的操作易于实现。

处理冲突的方法

• 哈希表链表的存储结构

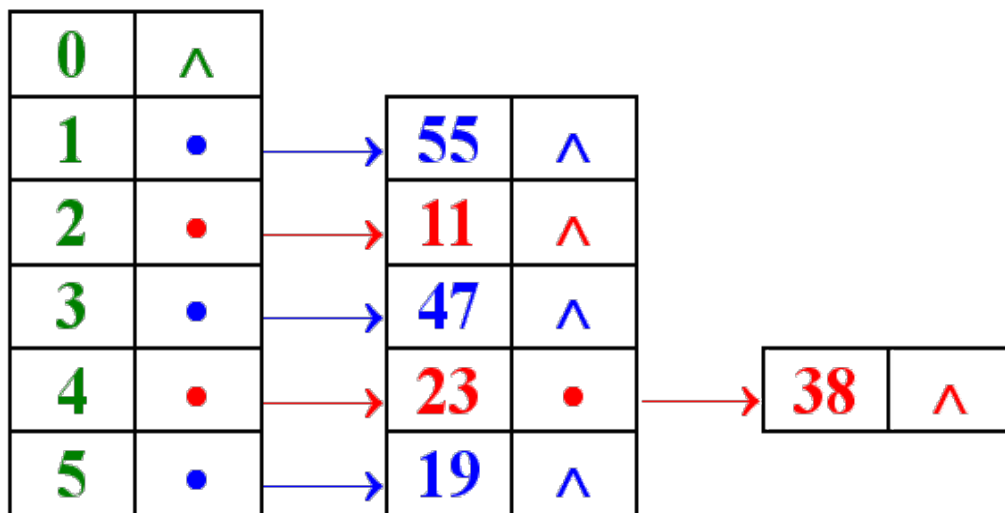
```
#define TABLE_SIZE 20
typedef struct HashNode {
    int key;
    void* data;
    struct HashNode* next;
} HashNode;
typedef struct {
    HashNode* heads[TABLE_SIZE];
    int count;
} HashTable;
```

// 链表节点结构（存储同义词）
// 关键字
// 指向实际数据记录的指针（泛型）
// 指向下一个同义词的指针

// 哈希表结构（表头数组）
// 指针数组，每个元素指向链表头节点
// 表中记录总数

处理冲突的方法

- 例 关键字集={19, 23, 55, 11, 47, 38}，表长m=6。
- 取哈希函数 $H(\text{key})=\text{key}\%5+1$ ，
- 则哈希地址={5, 4, 1, 2, 3, 4}



哈希表查找算法：开放地址法

• 算法

– 计算哈希值

```
int start = hash(key);
```

– 计算下一个哈希改变量（线性探测为例）直至遍历

```
for (step = 0; step < TABLE_SIZE; ++step) {
```

– 算出新哈希

```
int index = (start + step) % TABLE_SIZE;
```

– 如果该位置为空，找不到

```
if (table[index].state == EMPTY)  
    return -1;
```

– 如果找到了，返回

```
if (table[index].state == OCCUPIED  
    && table[index].key == key)  
    return index;
```

– 默认找不到

```
}  
return -1;
```

```

typedef struct {
    int key;
    int state; /* EMPTY、OCCUPIED 或 DELETED */
} OpenAddressSlot;

int searchOpenAddress(const OpenAddressSlot table[], int key) {
    int start = hash(key);
    int step;
    for (step = 0; step < TABLE_SIZE; ++step) {
        int index = (start + step) % TABLE_SIZE;
        if (table[index].state == EMPTY)
            return -1;
        if (table[index].state == OCCUPIED
            && table[index].key == key)
            return index;
    }
    return -1;
}

```


哈希表查找算法：链地址法

- 算法

- 计算哈希值
- 进入链表，直到找到关键字，否则进入下一节点

```
ChainNode* searchChained(const ChainedHashTable* table, int key) {  
    ChainNode* current = table->buckets[hash(key)];  
    while (current != NULL) {  
        if (current->key == key)  
            return current;  
        current = current->next;  
    }  
    return NULL;  
}
```

哈希表查找算法：性能分析

- 哈希表的平均查找长度实际上并不等于1。
- 决定哈希表平均查找长度ASL的因素：
- 处理冲突的方法和装填因子。
- 例如，对于表长 $m=6$ ，关键字集
- $= \{ 19, 23, 55, 11, 47, 38 \}$ ，
- 如果采用链地址法， $ASL = (1 \times 5 + 2 \times 1) / 6 > 1$
- 一般情况下，装填因子 α 越小，发生冲突的可能性越小；装填因子 α 越大，填记录就越容易发生冲突。

哈希表查找算法：性能分析

- 哈希表的一个特点：

- 用哈希表构造列表时，可以选择一个适当的装填因子 α ，使得平均查找长度限定在某个范围内。一般情况下，哈希表的平均查找长度是 α 的函数。如利用线性探测再散列处理冲突法的平均查找长度 $ASL = \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$ 。

哈希表查找算法：性能分析

- 例 已知列表L中的1000个关键字各不相同。请给出列表L的一个哈希表设计方案，要求它在等概率情况下，查找成功时的平均查找长度不超过3。
 - (1) 选择处理冲突的方法，并求出 α
 - “线性探测再散列”处理冲突法。由于要求 $ASL \leq 3$ ， $\alpha \leq 0.8$
 - (2) 确定哈希表的长度m
 - 一般取 $m \geq k/\alpha$ 中的最小素数。
 - $\because k/\alpha = 1000/0.8 = 1250$
 - $\Rightarrow m = 1259$ (根据给定的素数表找出满足要求的最小质数)。

本节小结

- 哈希表概念
- 哈希函数：确定关键字和存储位置对应关系的函数 $\text{Hash}(\text{key})$ ，其构造方法主要有直接定址法和除留余数法等。
- 处理冲突方法：当出现冲突时，为关键字寻找下一个存储位置的方法。主要方法有开放定址法和链地址法等。
- 哈希表的查找和性能分析。

8

谢谢观看

理论课程



厦門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang